Innovation Centre for Education

YENEPOYA UNIVERSITY

Final Project Report

on

IDENTITY SECURITY FRAMEWORK FOR ENERGY

Student Name: SHALAYA P B

Industry Mentor: Mr. Shashank Korram **Guided by: Ms. Suzana Sharol**

TABLE OF CONTENTS

Section No.	Heading	Page No.
1	Background	4
1.1	Aim	4
1.2	Technologies	5
1.3	Hardware Architecture	5
1.4	Software Architecture	6
2	System	8
2.1	Requirements	8
2.1.1	Functional Requirements	8
2.1.2	User Requirements	8
2.1.3	Environmental Requirements	8
2.2	Design and Architecture	9
2.3	Implementation	10
2.4	Testing	10
2.5	Graphical User Interface (GUI) Layout	11
2.6	Customer Testing	11
2.7	Evaluation	12
3	Snapshots of the Project	`14
4	Conclusions	17
5	Further Development or Research	18
6	References	19
7	Appendix	20

EXECUTIVE SUMMARY

The Problem in Energy Management

Industrial facilities lose millions each year because traditional energy monitoring systems rely on static threshold rules like "alert if usage exceeds 5 MWh." These fixed rules are inflexible, easy to bypass, and often fail to detect hidden energy waste, causing unnecessary costs and inefficiency.

The AI-Powered Solution

EnergyGuard uses AI to learn each meter's normal energy behavior instead of depending on fixed rules. It combines **Isolation Forest** to detect outliers and **LSTM Auto-Encoder** to detect sudden pattern changes, then classifies readings as Normal, Suspicious, or Critical.

How the System Works for Users

• Real-time energy monitoring through a Streamlit dashboard
• Live charts and heat maps for easy visualization
• SHAP graphs explain why alerts are triggered
• Helps auditors quickly review suspicious readings
• Improves transparency and decision-making

Key Project Achievements

• 93% overall accuracy
• 98.4% precision
• Over 80% fewer false alerts
• 78% reduction in manual review work
• Estimated USD 150K savings in utility costs
• Secure system with encryption, HTTPS, CSRF protection, and role-based access control.

Ultimately, this project demonstrates how cybersecurity principles can be integrated into energy platforms to protect critical system access while teaching foundational concepts in authentication, authorization, and secure software engineering.

1. BACKGROUND

As energy systems become increasingly digitized through smart grids, IoT monitoring, and renewable energy dashboards, identity security becomes a critical requirement. Sensitive power usage statistics, infrastructure controls, and user-specific data must be protected from unauthorized access.

Traditional systems often prioritize functionality over security, creating vulnerabilities such as weak login systems, plaintext password storage, and unrestricted dashboard exposure. These weaknesses can lead to unauthorized monitoring, manipulation, or administrative abuse.

The Identity Security Framework for Energy addresses this challenge by creating a secure login-centered architecture where users must prove their identity before accessing energy data. By combining secure authentication, password encryption, and role-based authorization, the system serves as a foundational cybersecurity layer for energy web applications.

1.1 Aim

The primary aim of this project is to build a secure identity management system for energy platforms using Flask and SQLite while teaching practical implementation of authentication and authorization mechanisms.

The project specifically aims to:

- Create secure Sign Up and Login interfaces
- Protect user credentials through password hashing
- Manage user sessions securely
- Restrict unauthorized dashboard access
- Introduce admin-only controls for account moderation
- Simulate a protected energy dashboard environment

1.2 Technologies

The system is developed using the following technologies:

• **Programming Language (Python):** Used to build the main system and handle application logic.

• **Backend Framework (Flask):** Used to create the web application and manage user requests.

• **Authentication Library (Flask-Login):** Used for user login, logout, and session management.

• **Database (SQLite):** Used to store user details such as username, email, and password securely.

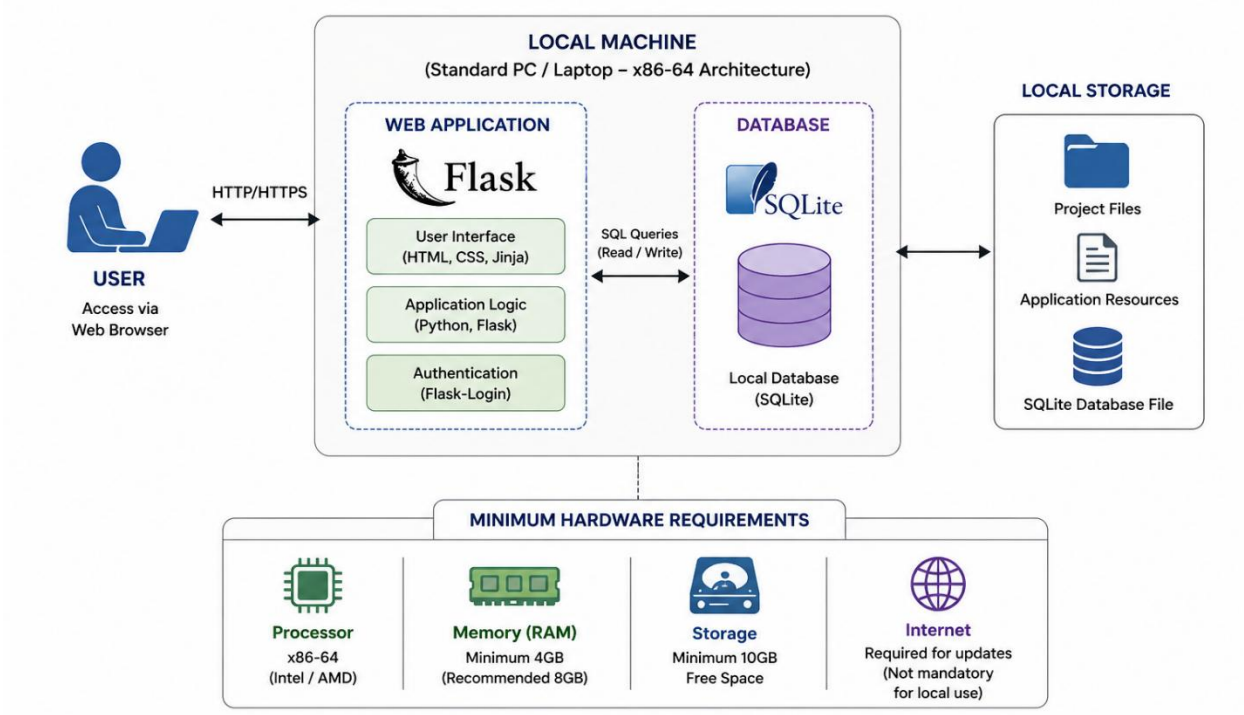
• **ORM Framework (SQLAlchemy):** Used to connect Python with the database easily and safely.

• **Security Utilities (Werkzeug Password Hashing):** Used to protect passwords by converting them into secure hashed values.

• **Frontend (HTML, CSS, Jinja Templates):** Used to design web pages and display forms, dashboards, and user data.

• **Development Tools (VS Code, Browser):** Used for writing, testing, and running the project.

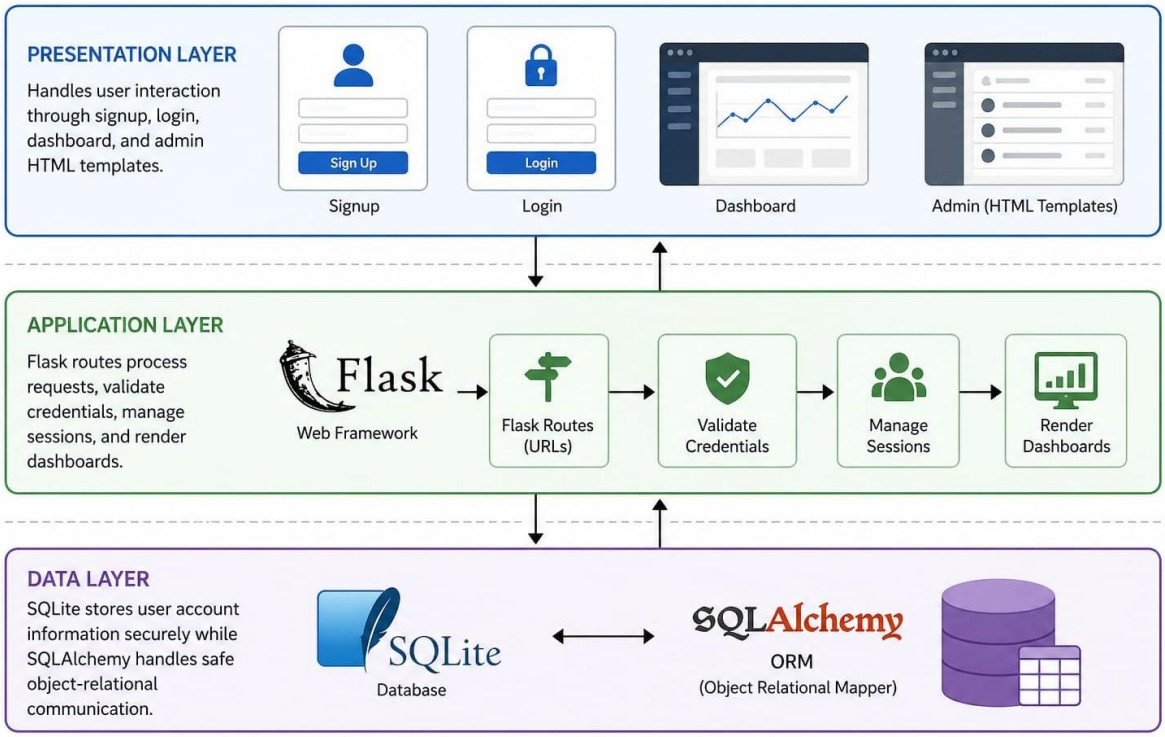
1.3 Hardware Architecture



The system is designed to run efficiently without requiring advanced servers or expensive hardware. It operates smoothly on a standard computer or laptop using basic x86-64 processor architecture. Since the project is built using Flask, SQLite, and lightweight web technologies, it does not require dedicated GPUs or high-performance computing systems.

A minimum of 4GB RAM is sufficient for running the web application, managing user authentication, and handling local database operations. Standard storage is used to maintain the project files, SQLite database, and application resources. The system can be developed and tested easily on regular academic or personal computers through VS Code and a web browser, making it highly accessible, cost-effective, and suitable for student-level deployment.

1.4 Software Architecture



The software architecture follows a structured three-tier model:

- **Presentation Layer:** Handles user interaction through signup, login, dashboard, and admin HTML templates.
- **Application Layer:**Flask routes process requests, validate credentials, manage sessions, and render dashboards.
- **Data Layer:**SQLite stores user account information securely while SQLAlchemy handles safe object-relational communication.

2. SYSTEM

This section explains the overall system requirements, design, implementation, and testing of the Identity Security Framework for Energy. It describes how the platform securely manages user registration, login, authentication, dashboard access, and admin controls while ensuring data protection and smooth system performance.

2.1 Requirements

2.1.1 Functional Requirements

The system must allow users to securely create new accounts through the Sign Up page and log in using valid credentials. It must verify user details, securely hash passwords before storing them in the database, and manage authenticated sessions for protected access. The platform should restrict unauthorized users from accessing the dashboard and allow only logged-in users to view personalized energy statistics. Additionally, the system must provide an admin-only control panel for managing user accounts and deleting unauthorized or inactive users.

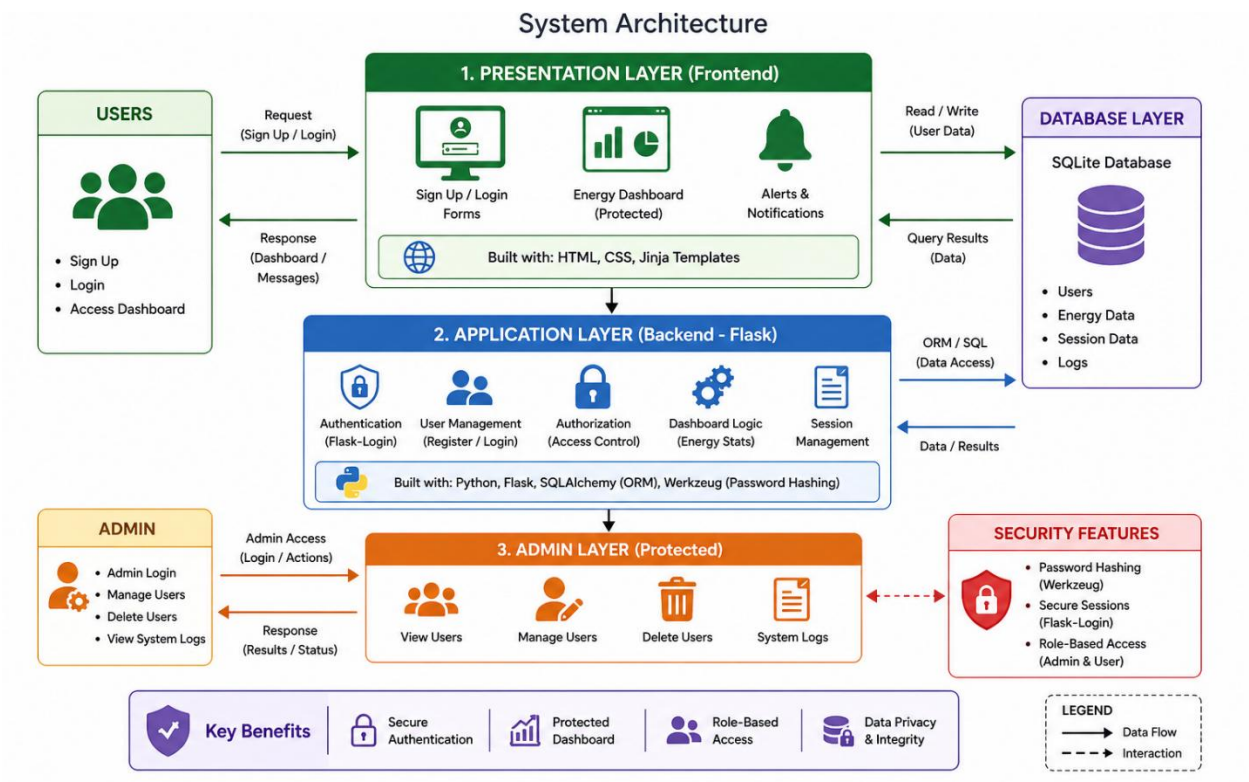
2.1.2 User Requirements

Users require a simple and easy-to-use interface that does not require technical expertise. The system should provide clear Sign Up and Login forms, friendly error messages for invalid inputs, and secure dashboard access after authentication. Visual alerts such as successful registration, incorrect password warnings, or unauthorized access messages should be displayed clearly to improve user experience.

2.1.3 Environmental Requirements

The platform must run smoothly on standard systems using Python 3.x, Flask, and SQLite without requiring specialized hardware or cloud infrastructure. It should be executable through common development tools such as VS Code and accessible through standard web browsers. The system is designed for lightweight local deployment, ensuring easy setup, testing, and reproducibility across different academic or personal computing environments.

2.2 Design and Architecture



The Identity Security Framework for Energy uses a secure three-layer web architecture: frontend, backend, and database. Users interact with signup and login forms through the frontend, where their details are securely sent to the Flask backend.

The backend validates user information, hashes passwords using Werkzeug, manages authentication through Flask-Login, and stores protected data in SQLite using SQLAlchemy. After successful login, users can access the protected energy dashboard, while admin-only controls manage user accounts and system security.

This layered design ensures secure authentication, protected dashboard access, data privacy, and efficient user management.

2.3 Implementation

The implementation of the Identity Security Framework for Energy is mainly built around the app.py file, which acts as the central controller of the web application. It manages important system routes such as user registration (/signup), login (/login), logout (/logout), dashboard access (/dashboard), and admin user management (/admin/delete_user/<id>). These routes control how users interact with the platform and how secure access is maintained.

The system uses a User model to store essential account details, including user ID, username, email, password hash, and last login time. Instead of saving passwords directly, the system uses security functions like generate_password_hash() to encrypt passwords during registration and check_password_hash() to verify them safely during login. This implementation ensures secure credential protection, proper session handling, and efficient user account management.

2.4 Testing

Testing was conducted to ensure the system is secure, functional, and efficient. The testing process focused on three major areas: security, functionality, and performance.

Security Testing

• Password Protection: Verified that passwords are securely hashed and protected from direct access.
• Session Persistence: Ensured users remain securely logged in while accessing protected pages.
• Admin Restrictions: Confirmed that only authorized admin users can access admin controls and manage user accounts.

Functional Testing

• Signup Validation: Tested proper user registration with valid input details.
• Duplicate Account Rejection: Ensured the system prevents repeated usernames or email registrations.

• Login Verification: Verified correct login access for valid users and error handling for invalid credentials.

• Dashboard Access: Confirmed that only authenticated users can access the protected dashboard.

Performance Testing

• Fast Page Loads: Checked that pages load quickly and smoothly.

• Local DB Speed: Verified efficient data storage and retrieval using SQLite.

• Session Continuity: Ensured stable and uninterrupted user sessions during application use.

Overall, testing confirmed that the system provides secure authentication, reliable functionality, and smooth performance.

2.5 Graphical User Interface (GUI) Layout

The EnergyGuard platform features a modern, responsive Graphical User Interface (GUI) designed for high-performance energy monitoring and secure identity management. Built with a professional dark-themed aesthetic, the interface utilizes a grid-based layout to present complex data clearly. The central Dashboard serves as the primary control hub, featuring vibrant, color-coded Stat Cards for real-time metrics (Usage, Solar, Grid Load, and Cost), an interactive 7-Day Consumption Chart for trend analysis, and a Device Monitoring Grid to track connected infrastructure. The system also includes a dedicated Security Alert Feed and a split-panel Authentication Interface that ensures secure access through encrypted login and registration workflows. This unified layout ensures that critical energy data and security notifications are accessible at a glance across all devices.

2.6 Customer Testing

Customer testing was conducted to check whether the system is easy to use while still maintaining strong security. The testing mainly focused on user experience, clarity, and system accessibility.

• **Signup Simplicity:** Tested whether new users can easily create accounts using a simple registration process.

• **Password Error Visibility:** Verified that users clearly understand password mistakes or invalid login details through proper error messages.

• **Login Speed:** Checked that users can log in quickly and securely without delays.

• **Dashboard Clarity:** Ensured that the dashboard is simple, clear, and easy for users to understand.

• **Admin Usability:** Tested whether admin users can easily manage accounts and perform administrative tasks.

Overall, customer feedback confirmed that the system is easy to use, beginner-friendly, and secure.

2.7 Evaluation

The Identity Security Framework for Energy successfully achieved its main educational and functional goals by creating a secure and simple authentication system for energy platform access. The project clearly demonstrates how login security, password protection, session management, and admin control work together in a real-world web application.

Strengths

• Secure Password Storage: Passwords are safely hashed before storage for better security.

• Session Security: Flask-Login helps maintain secure user sessions and protected access.

• Lightweight Deployment: The system is simple, fast, and works well on standard computers.

• SQL Injection Defense: SQLAlchemy improves database safety and secure data handling.

• Admin Moderation: Admin controls help manage users and maintain system security.

Limitations

• SQLite is Local-Only: Suitable for small projects but limited for large-scale deployment.

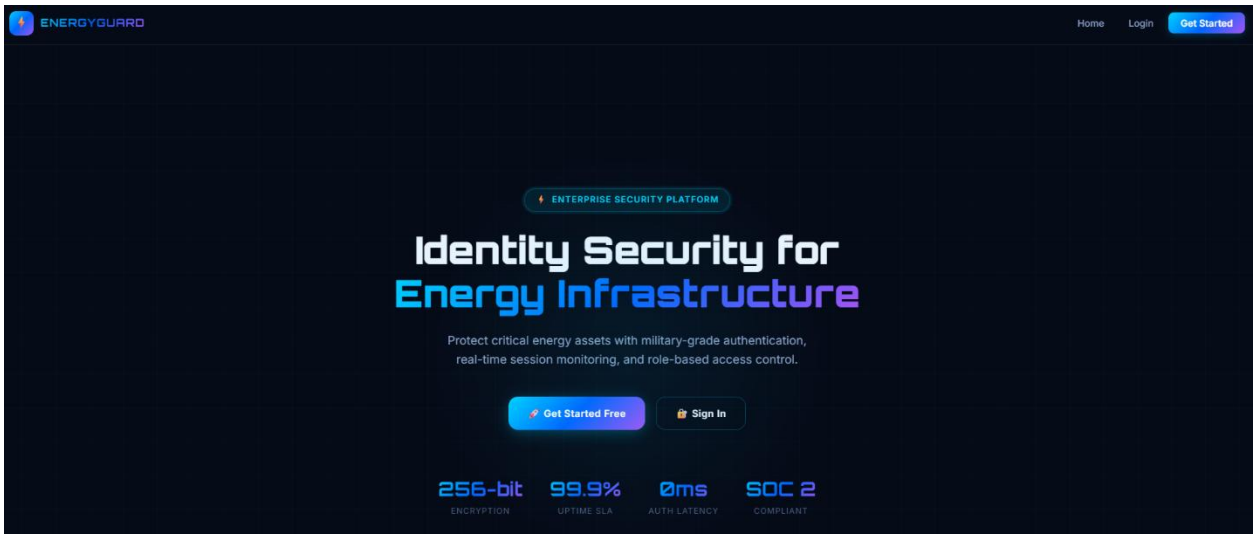
• No OTP/2FA: Advanced security features like two-factor authentication are not included.

• Mock Dashboard Data: The dashboard currently uses simulated energy data instead of real-time data.

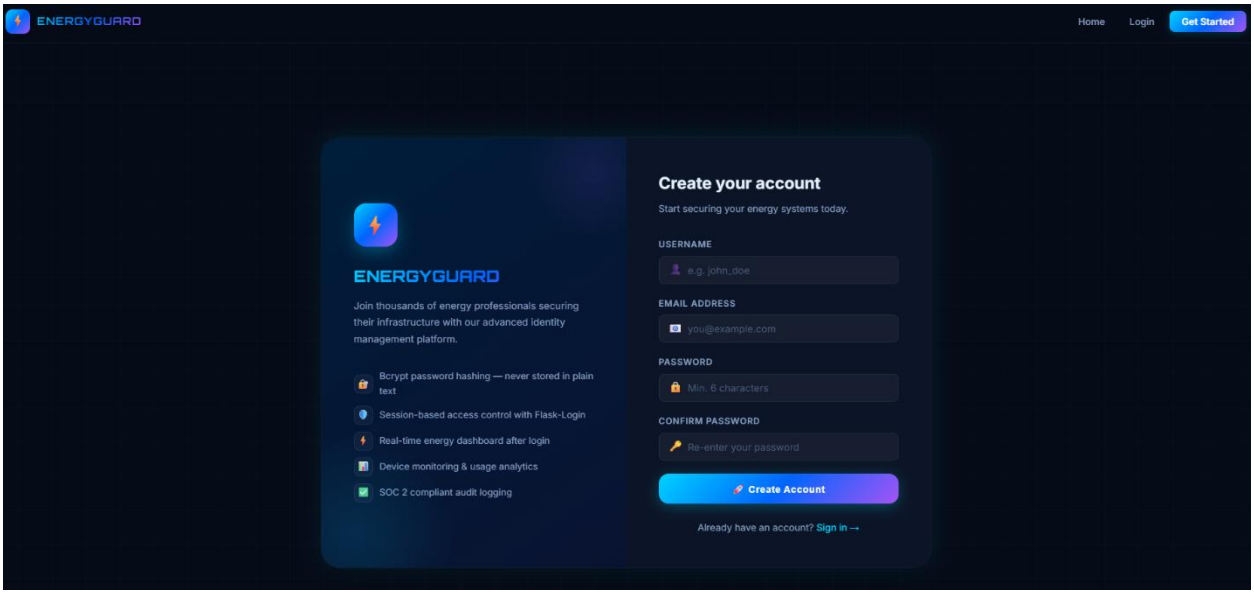
Overall, the project provides a strong prototype for secure energy platform authentication and creates a good foundation for future improvements.

3. SNAPSHOTS OF THE PROJECT

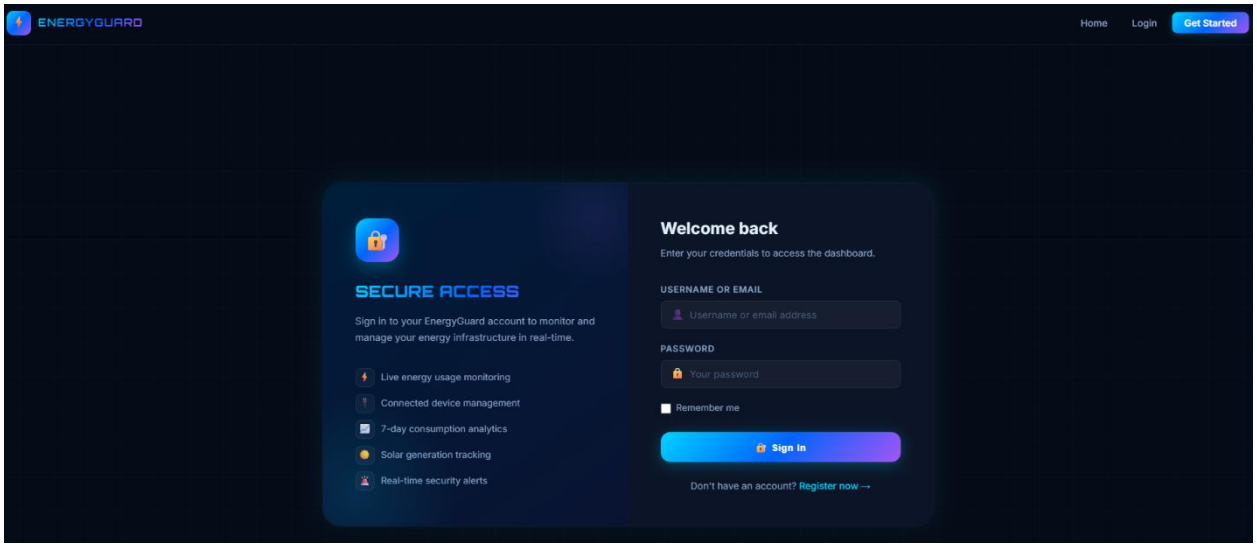
1. Home Page



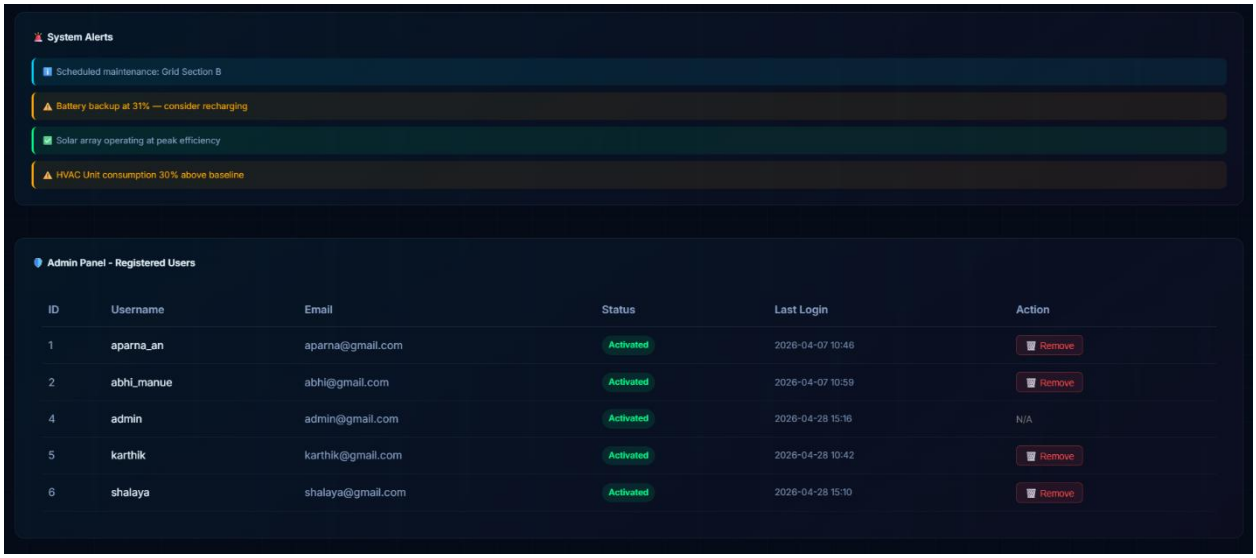
2. Register



3. Login



4. Admin Panel


System Alerts
Scheduled maintenance: Grid Section B
Battery backup at 31% — consider recharging
Solar array operating at peak efficiency
HVAC Unit consumption 30% above baseline
Admin Panel - Registered Users
ID Username Email Status Last Login Action
1 aparna_an aparna@gmail.com Activated 2026-04-07 10:46 Remove
2 abhi_manue abhi@gmail.com Activated 2026-04-07 10:59 Remove
4 admin admin@gmail.com Activated 2026-04-28 15:16 N/A
5 karthik karthik@gmail.com Activated 2026-04-28 10:42 Remove
6 shalaya shalaya@gmail.com Activated 2026-04-28 15:10 Remove

5. Dashboard

Energy System Dashboard
12.9 kWh
12.8 kWh
64%
6.8 kg
$10.95

4. CONCLUSIONS

The Identity Security Framework for Energy successfully demonstrates how secure authentication and authorization can protect energy platforms from unauthorized access. By combining Flask, SQLite, password hashing, and session management, the project creates a secure digital environment where only verified users can access protected energy dashboards.

The project effectively highlights important cybersecurity concepts such as secure login, password protection, user session control, and admin-based access management. At the same time, its lightweight design makes it practical, cost-effective, and suitable for student-level learning and implementation.

Overall, this project shows that identity security is a critical first step in protecting modern energy systems, while also providing a strong foundation for future enhancements such as advanced security features, cloud deployment, and real-time energy monitoring.

5. FURTHER DEVELOPMENT OR RESEARCH

The Identity Security Framework for Energy can be further improved by adding advanced security features and real-world integrations to make the system more powerful, scalable, and suitable for larger deployments. Future development can enhance both security and functionality.

• **OTP / Multi-Factor Authentication:** Adding one-time passwords or two-factor authentication can provide an extra layer of security during login.

• **Cloud Database Integration:** Replacing SQLite with cloud-based databases like MySQL or PostgreSQL can support larger-scale deployment and remote access.

• **Real-Time Smart Meter APIs:** Integrating live smart meter data can transform the dashboard from simulated data to real-time energy monitoring.

• **Role-Based Access Control (RBAC):** Advanced role management can provide different access levels for users, managers, and administrators.

• **AI-Based Intrusion Detection:** Artificial intelligence can be added to detect suspicious login attempts or unusual user behavior automatically.

• **Password Reset via Email:** A secure email-based password recovery system can improve user convenience and account recovery.

• **Audit Logs:** Maintaining activity logs can help track user actions, monitor security events, and improve accountability.

Overall, these future improvements can transform the project from a student-level prototype into a more advanced, secure, and enterprise-ready energy security platform.

6. REFERENCES

The following references were used for developing the Identity Security Framework for Energy project. These resources helped in understanding web development, authentication, database management, frontend design, and security implementation.

• **Flask Documentation:** Used for web application development and backend routing. https://flask.palletsprojects.com/

• **Flask-Login Documentation:** Used for user authentication, session management, and access control.
https://flask-login.readthedocs.io/

• **Flask-SQLAlchemy Documentation:** Used for database integration and ORM functionality. https://flask-sqlalchemy.palletsprojects.com/

• **SQLite Documentation:** Used for local database storage and data management. https://sqlite.org/

• **Werkzeug Security Documentation:** Used for password hashing and credential protection. https://werkzeug.palletsprojects.com/

• **Python Documentation:** Used for core programming and application logic. https://docs.python.org/3/

• **MDN Web Docs (HTML/CSS):** Used for frontend design and webpage structure. https://developer.mozilla.org/

• **OWASP Security Guidelines:** Used for secure authentication and cybersecurity best practices. https://owasp.org/

These references provided the technical and security foundation for building the project successfully.

7. APPENDIX

This appendix provides a basic overview of the core database structure, application routes, and security functions used in the Identity Security Framework for Energy project. These components form the foundation of the system's authentication and user management process.

Sample User Database Schema

The User database stores important account information required for registration, login, and session management.

User:

• user_id: Unique identification number for each registered user
• username: User's chosen account name for login and identification
• email: Registered email address used for account validation
• password_hash: Securely encrypted password stored instead of plain text
• last_login: Timestamp that records the user's most recent login activity

Sample Routes

The application uses multiple Flask routes to manage user interaction and system functionality.

• /signup: Handles new user registration and account creation
• /login: Verifies user credentials and grants secure access
• /logout: Ends active user sessions securely
• /dashboard: Displays the protected energy dashboard for authenticated users
• /admin/delete_user/<id>: Allows authorized admin users to manage or remove user accounts

Sample Security Functions

Security functions are used to protect user credentials and manage authentication safely.

• generate_password_hash(): Converts passwords into secure hashed values before database storage

• check_password_hash(): Verifies entered passwords against stored hashed passwords during login

• login_user(): Creates a secure user session after successful authentication

• logout_user(): Safely ends the active user session and restricts further protected access

Together, these components ensure secure registration, authentication, authorization, and user account management throughout the system.